

CRIAÇÃO DE UMA APLICAÇÃO FRONT-END PARA UMA API DE DESENHOS DE SUCESSO DOS ANOS 90 E 2000

1. O que é uma API e como funciona o consumo de dados em tempo real

Uma **API (Application Programming Interface)** atua como uma ponte de comunicação entre diferentes sistemas de software. No contexto do desenvolvimento web front-end, uma API RESTful (como a utilizada neste projeto) permite que uma aplicação cliente (o navegador do usuário) solicite dados de um servidor remoto. O servidor processa a requisição e devolve as informações, geralmente no formato estruturado **JSON** (JavaScript Object Notation).

O **consumo de dados em tempo real** (ou dinâmico) ocorre quando a aplicação web não possui os dados "chumbados" (hardcoded) no seu código-fonte HTML. Em vez disso, no momento em que o usuário acessa a página, o JavaScript realiza uma requisição "por debaixo dos panos" para o servidor da API. Assim que os dados mais recentes são recebidos, a interface gráfica é renderizada com as informações atualizadas. Isso garante que qualquer alteração no banco de dados do servidor seja refletida instantaneamente para o usuário final, sem a necessidade de atualizar o aplicativo inteiro.

2. Conceito de DOM Manipulation e a Criação de Elementos Dinâmicos

O **DOM (Document Object Model)** é uma representação estruturada do documento HTML em formato de árvore de nós (nodes). Cada elemento da página (como um `<div>`, um `<p>` ou uma imagem) é um objeto no DOM que o JavaScript consegue acessar, modificar, deletar ou criar. A **criação de elementos dinâmicos** é o processo onde a interface é construída via código JavaScript em vez de ser escrita fisicamente em um arquivo .html. No nosso projeto, o arquivo HTML contém apenas um contêiner vazio (`<div id="characters-container"></div>`). O JavaScript

percorre os dados recebidos da API e, para cada personagem, fabrica uma nova estrutura de tags HTML na memória do navegador. Após montar essa estrutura (com a imagem, nome e status), o JavaScript injeta esses novos elementos dentro do contêiner vazio no DOM. Isso torna a aplicação escalável: independentemente de a API retornar 10 ou 100 personagens, o código JavaScript lidará com a renderização de forma automática.

3. Funções Básicas para Consumo e Manipulação

Para que a magia da requisição e renderização aconteça, o projeto faz uso de um conjunto de funções nativas do JavaScript Moderno. Abaixo detalhamos as principais:

Função / Método	Descrição e Aplicação no Projeto
fetch()	Função global utilizada para realizar requisições de rede (HTTP). No projeto, ela é chamada como <code>fetch(API_URL)</code> para buscar os dados no servidor do Rick and Morty. Ela retorna uma <i>Promise</i> .
async / await (e .then)	Para lidar com a assincronicidade (o tempo que a API demora para responder), usamos <code>await</code> , que é uma sintaxe moderna baseada no método <code>.then()</code> . Isso pausa a execução do código naquela linha até que os dados cheguem, evitando erros de leitura em variáveis vazias.
response.json()	Os dados trafegam pela rede como texto puro. Este método transforma a resposta (texto) em um Objeto JavaScript real, permitindo acessar propriedades como <code>data.results</code> .
document.createElement()	Cria uma nova tag HTML na memória do navegador. No projeto, usamos

Função / Método	Descrição e Aplicação no Projeto
	<code>document.createElement('div')</code> para criar o "esqueleto" do card de cada personagem.
<code>appendChild()</code>	Pega um elemento gerado na memória (como o card criado acima) e o anexa fisicamente ao DOM visível. É através de <code>container.appendChild(card)</code> que os personagens finalmente aparecem na tela do usuário.

4. Motivo da Escolha da API: Rick and Morty

A escolha da API do **Rick and Morty** deu-se por razões fortemente ligadas ao desenvolvimento de front-end educacional e prático:

- **Riqueza Visual e de Dados:** A API fornece não apenas nomes, mas dados categóricos ricos (espécie, origem, localização atual) e imagens em alta qualidade pré-hospedadas. Isso permite a construção de uma interface atraente e de componentes complexos, como a "Gaveta Lateral" (Side Drawer) implementada no projeto.
- **Prontidão para Tipagem:** Os atributos bem definidos (como status: "Alive" ou "Dead") permitem praticar renderização condicional avançada no CSS e JS (como feito com a variação das cores das *status badges*).

5. Regras de Acesso e Documentação da API

Com base na documentação oficial da API utilizada no projeto, destacam-se as seguintes diretrizes e arquiteturas de acesso:

- **Autenticação (Tokens):** Trata-se de uma API pública e de código aberto. **Não é necessário o uso de tokens de autenticação**, chaves de API (API Keys) ou registros para consumir seus dados. Isso facilita o deploy direto em páginas estáticas.
- **Limites de Requisição (Rate Limits):** Embora seja aberta, a API implementa rate limits para evitar sobrecarga (DDoS ou abusos acidentais). O limite atual é de aproximadamente **10.000 requisições por dia** por endereço IP, o que é vastamente suficiente para aplicações de portfólio e escopo acadêmico.

- **Estrutura de Resposta (Payload):** A API responde com estrutura de dados no padrão REST. O *endpoint* base (/api/character) não retorna um array direto, mas sim um objeto complexo dividido em dois grandes nós:
 1. **info:** Contém metadados essenciais para paginação (contagem total de personagens, número de páginas, e URLs para next e prev).
 2. **results:** O Array contendo de fato os objetos dos personagens. É justamente devido a essa estrutura arquitetural que, no código, o mapeamento ocorre através de `data.results` e não diretamente na variável `base`.